

O projeto consiste em um jogo de sobrevivência, onde um jogador controla um sobrevivente que precisa administrar seus recursos e condições físicas para permanecer vivo.

O jogo conta com um cadastro de sobreviventes, onde são definidos nomes, idades e classes. Durante a aventura, o personagem possui quatro atributos principais, sendo eles **vida, fome, sede e sono**. Esses atributos são afetados pela passagem do tempo e pelas ações realizadas pelo jogador.

As principais funcionalidades são:

- Coleta de recursos como madeira, pedra, água e comida;
- Sistema de inventário;
- Caça e obtenção de carne;
- Combate contra animais;
- Construção de objetos, como machado, lança, fogueira e abrigo;
- Alimentação, hidratação e descanso;
- Passagem de dias e horas;
- Eventos aleatórios, como tempestades, ferimentos, descobertas e encontros com animais;
- Sistema de morte quando a vida chega a zero.

O programa utiliza uma matriz de regiões, como florestas, rios, campos e montanhas, contendo recursos diferentes entre si.

O desenvolvimento começou com a definição das principais mecânicas do jogo, principalmente os sistemas de vida, fome, sede e sono. Em seguida, foi criado o cadastro de sobreviventes e o sistema de inventário.

Em seguida foram adicionados os sistemas de coleta, caça, combate e passagem de tempo. Com essas funções funcionando, pode ser implementada a mecânica de construção de itens.

Por fim, foram adicionados os eventos aleatórios e a matriz de regiões. O código foi dividido em diversas funções, cada uma responsável por uma tarefa específica, facilitando sua organização e manutenção.

Um dos principais problemas foi organizar as diversas funcionalidades sem deixar o código confuso. Para resolver isso, o programa foi dividido em funções específicas, como `cacar()`, `combate()`, `construir()` e `coletar_recursos()`.

Também foi necessário controlar os valores de vida, fome, sede e sono para evitar que ultrapassassem os limites definidos, então foram adicionadas verificações para manter esses valores entre zero e cem.

Outro problema ocorreu nas entradas fornecidas pelo usuário. Valores inválidos poderiam causar erros ou interromper o programa. Para solucionar isso, foram utilizadas verificações e estruturas `try/except` para tratar entradas numéricas incorretas.

No sistema de construção, também foi necessário verificar se o jogador possuía materiais suficientes antes de fabricar um item. Foi criada uma função específica para realizar essa verificação.

Além do conteúdo das aulas, foi utilizado o W3CSchools e vídeos para compreender melhor os conceitos do Python durante o desenvolvimento.